E-Commerce Marketplace Application Design

GitHub Repository: https://github.com/Aadit-Mathur/DBMS-Project-Group-27

1. Introduction:

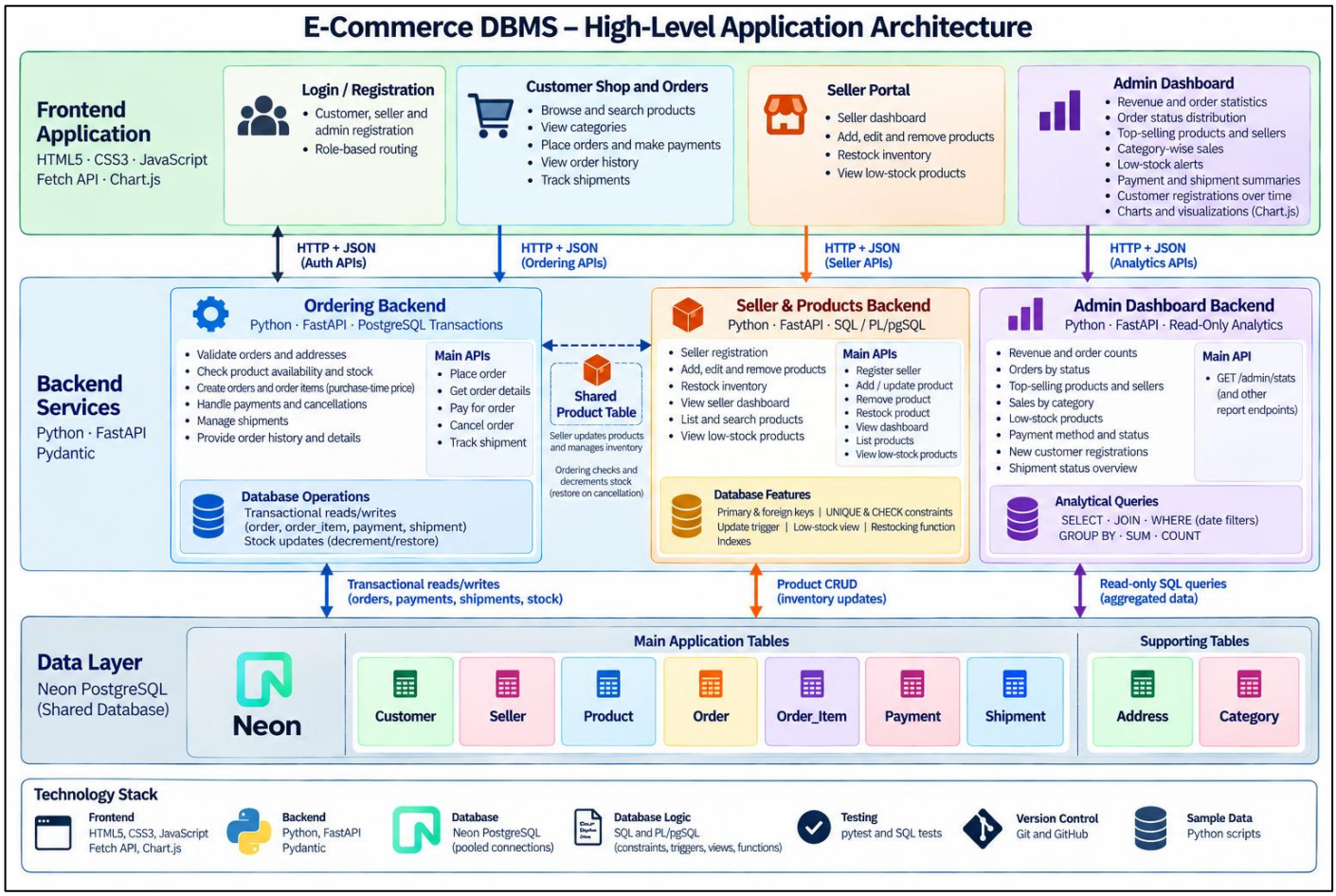
We are designing an E-Commerce platform, which allows sellers to list their products and customers to browse and place orders, much like Amazon.

The details of the database are elaborated in the Database Design Document.

2. System Overview and Architecture:

Our e-commerce application is a web-based platform consisting of three functional modules: Ordering Backend, Seller & Products Backend and Admin Dashboard Backend, along with the Frontend.

Below is the architecture diagram:



3. Technology Stack:

Component	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	Provides interfaces for customers, sellers, and administrators.
Charts and Visualization	Chart.js	Displays analytical statistics in the admin dashboard.
API Communication	JavaScript Fetch API, HTTP and JSON	Enables communication between the frontend and backend APIs.
Backend Framework	Python, FastAPI	Implements REST APIs for ordering, seller and product management, and admin analytics.
Request Validation	Pydantic	Validates backend request data.
Database	PostgreSQL hosted on Neon	Stores application data and enforces relational integrity.
Database Connectivity	psycopg2 and Neon pooled connections	Enables backend access to PostgreSQL.
Database Programming	SQL and PL/pgSQL	Implements queries, joins, aggregate reports, constraints, triggers, views, functions, and transactions.
Testing	pytest and SQL tests	Tests API behaviour, database operations, and integrity constraints.
Version Control	Git and GitHub	Supports collaborative development and source-code management.
Sample Data Generation	Python scripts	Populates the database with sample customers, sellers, categories, products, and related records.

4. Frontend:

4.1 Workflows:

The frontend provides a common web interface for customers, sellers, and administrators. The interface includes login and registration, a customer shopping interface, a seller portal, and an admin dashboard.

Customers can browse and search products, manage their shopping selections (via a client-side cart), check out, view orders, and track shipments. Sellers can manage products and inventory through their portal. Administrators can view platform-wide statistics and analytical reports.

4.2 Screens and API Integration;

Screen	Role	API integration
Login/Register	All roles	Login and registration APIs
Shop and Orders	Customer	Product search, category listing, order creation, order details and tracking
Seller Portal	Seller	Seller dashboard, product listing, add/edit/delete, restocking and low-stock APIs
Admin Dashboard	Administrator	Admin analytics API

5. Ordering Backend:

5.1 Workflows:

A customer places an order by selecting a delivery address and a list of products. The backend executes the operation as a PostgreSQL transaction. It validates that the address belongs to the customer, locks the relevant product rows, checks stock availability, records each item at its purchase-time price, decrements stock, and creates a pending payment. If any operation fails, the transaction is rolled back to prevent partial order creation.

The customer can subsequently pay for the order, cancel it, view order history, and track shipment status. Payment confirmation and shipment creation follow the order workflow. Cancellation restores the relevant stock according to the implemented business rules.

5.2 API Design:

Operation	Purpose
Place order	Validate order details and create an order with its items.
Get order details	Retrieve information about a particular order.
View order history	Retrieve a customer's previous orders.
Pay for an order	Process payment and update the relevant records.
Cancel an order	Cancel an eligible order and restore stock.
Track shipment	Retrieve shipment and delivery status.

5.3 Database Interactions:

The Ordering Backend reads customer, address, and product information and writes to the Order, Order_Item, Payment, and Shipment tables. PostgreSQL transactions ensure that related order operations are performed atomically. Row locking and stock validation help prevent concurrent orders from exceeding available inventory.

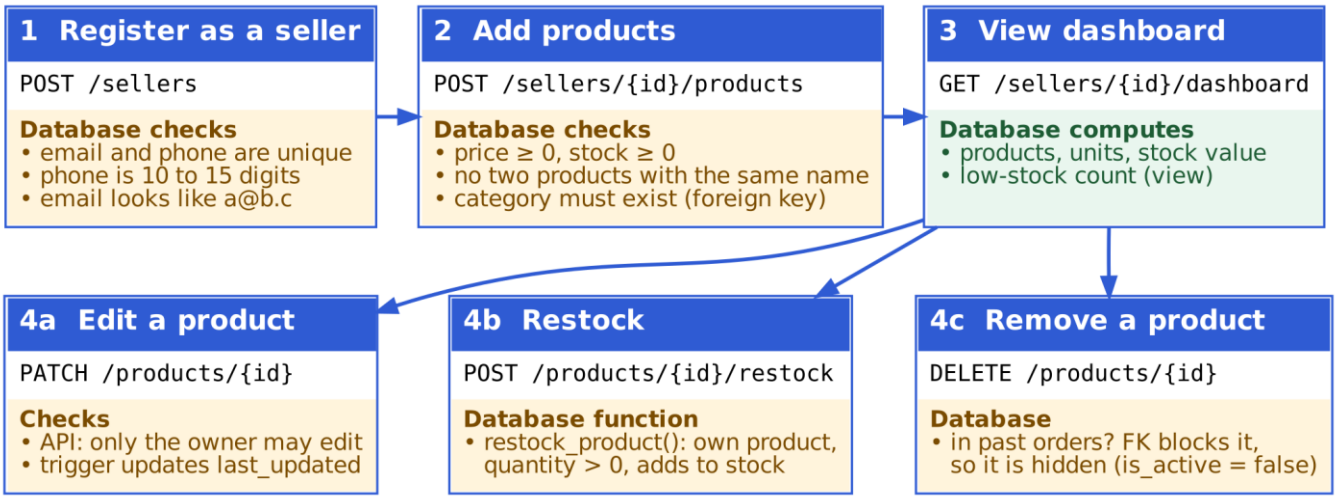
The Ordering Backend shares the Product table with the Seller & Products Backend. Order placement decreases stock, while eligible cancellations restore stock.

6. Seller & Products Backend:

6.1 Workflows:

A seller registers on the platform and lists products under existing categories. The seller can view a dashboard containing product and inventory statistics, edit product details, restock products, and remove or delist products.

PostgreSQL enforces data-integrity rules through constraints, foreign keys, triggers, views, and database functions. The backend translates database errors into appropriate HTTP responses so that invalid operations are reported clearly and do not result in invalid database changes.



6.2 API Design:

Endpoint	Purpose
POST /sellers	Register a seller.
POST /sellers/{id}/products	Add a product.
GET /sellers/{id}/dashboard	Retrieve seller-specific statistics.
GET /sellers/{id}/products	List a seller's products.
PATCH /products/{id}	Update product details.
POST /products/{id}/restock	Increase stock through the restocking operation.
DELETE /products/{id}	Remove or delist a product, subject to database rules.
GET /sellers/{id}/low-stock	Retrieve low-stock products.
GET /products?q=...	Search products.
GET /categories	Retrieve available categories.

6.3 Database Interactions:

The Seller & Products Backend interacts primarily with the Seller, Product, and Category tables. PostgreSQL constraints enforce data-integrity rules, including uniqueness, valid product prices, stock requirements, and category references. A trigger maintains the product's last-updated timestamp, a low-stock view identifies products requiring attention, and a database function performs restocking operations. Indexes support common product retrieval and search queries.

The Product table is shared with the Ordering Backend, which decreases stock during order placement and restores it on eligible cancellations. The two modules must therefore use consistent stock-management rules.

7. Admin Dashboard Backend:

7.1 Workflows:

The Admin Dashboard Backend provides administrators with a read-only analytical view of the e-commerce platform. It retrieves and aggregates information from the Customer, Seller, Product, Order, Order_Item, Payment, and Shipment tables. The frontend displays the resulting statistics through dashboard summaries, tables, and charts.

The dashboard covers revenue and order counts over a selected date range, order status distribution, top-selling products, top sellers, category-wise sales, low-stock alerts, payment breakdowns, customer registrations over time, and shipment status summaries.

7.2 API Design:

Operation	Purpose
Get admin stats	Retrieve the main dashboard statistics.
Revenue and order report	Retrieve revenue and order counts for a date range.
Order status report	Group orders by their status.
Product and seller report	Identify top-selling products and sellers.
Category sales report	Aggregate sales by category.
Low-stock report	List products where stock is at or below the reorder level.
Payment report	Summarize payments by method and status.
Customer registration report	Show new registrations over time.
Shipment report	Summarize shipments by status.

7.3 Database Interactions:

The Admin Dashboard Backend executes SQL queries directly against PostgreSQL to retrieve and aggregate platform data. The queries use joins, filtering, grouping, and aggregate functions such as SUM and COUNT to generate analytical reports.

The reports use the relevant relationships between customers, sellers, products, orders, order items, payments, and shipments. Low-stock alerts identify products satisfying $Quantity_In_Stock \leq Reorder_Level$. The reporting endpoints are read-only and do not modify transactional records.